

Here is the step-by-step Stack-Heap diagram analysis for the provided code.

I will use **Reference IDs** (e.g., @Ref1) to visualize how variables in the **Stack** point to objects in the **Heap**.

Legend

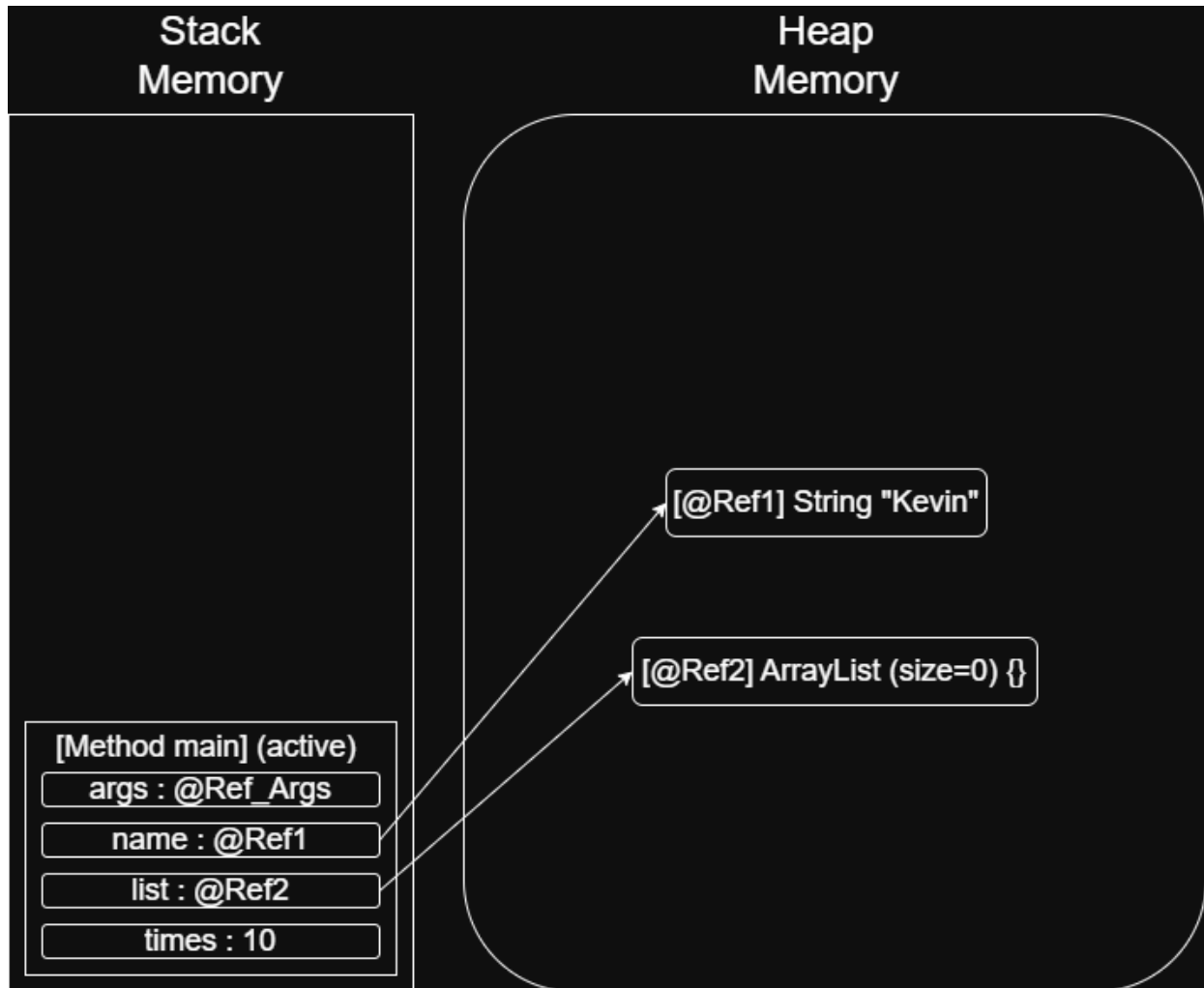
- **Stack:** Stores method frames, primitive variables (int, boolean), and references variables.
- **Heap:** Stores Objects (String, ArrayList, arrays).
- **Active Method:** The method currently executing at the top of the Stack.

Step 1: main Method Initialization

Execution:

1. String name = "Kevin" created.
2. List<String> list = new ArrayList<>() created.
3. int times = 10 created.

Diagram:

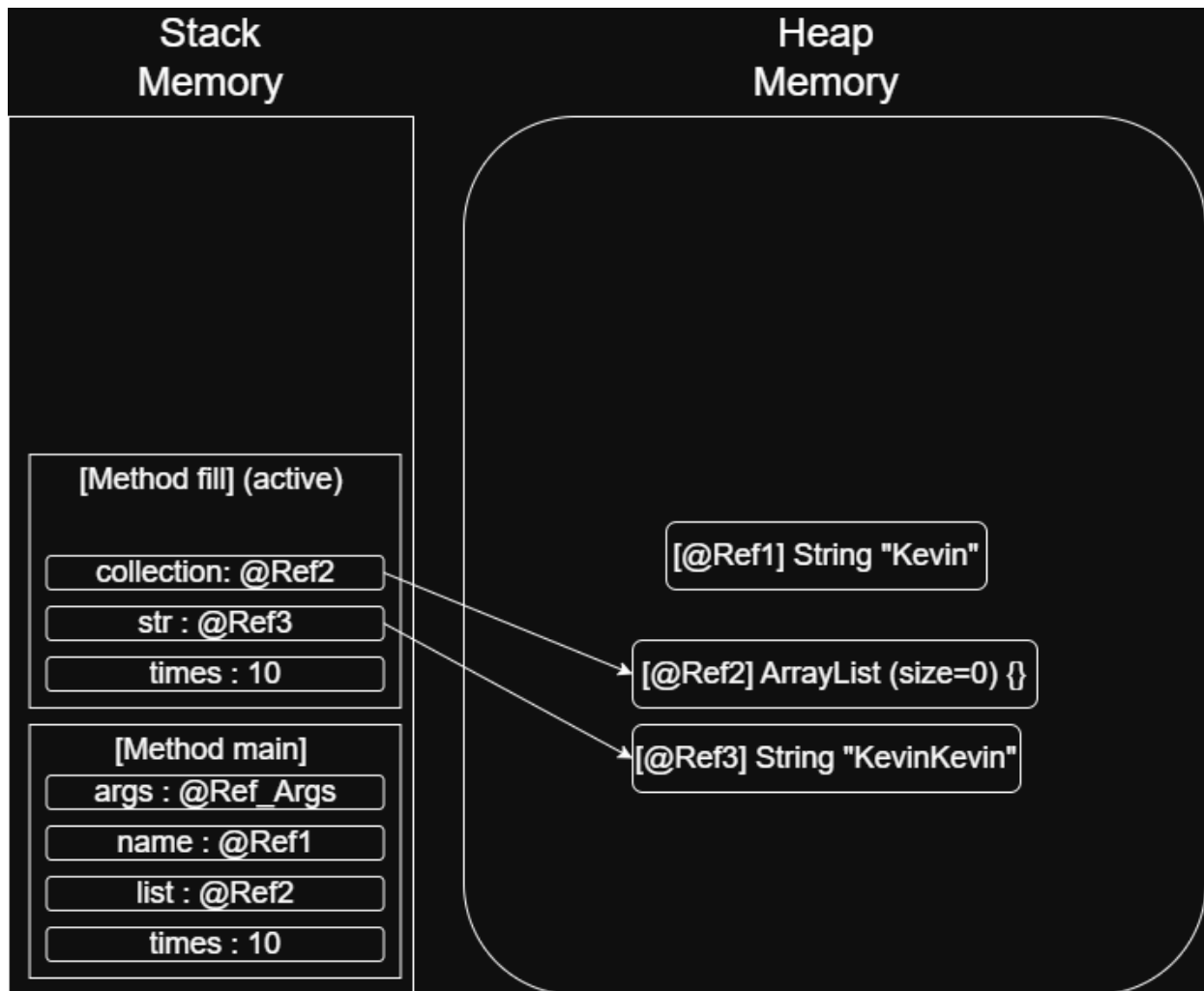


Step 2: Call to **fill** Method

Execution:

1. **name + name** is evaluated, creating a new String object **"KevinKevin"** in the Heap.
2. **fill** method is called.
3. Arguments are passed:
 - **collection** receives a copy of reference **@Ref2**.
 - **str** receives reference to the new string **"KevinKevin"**.
 - **times** (primitive) receives the value **10**.

Diagram:

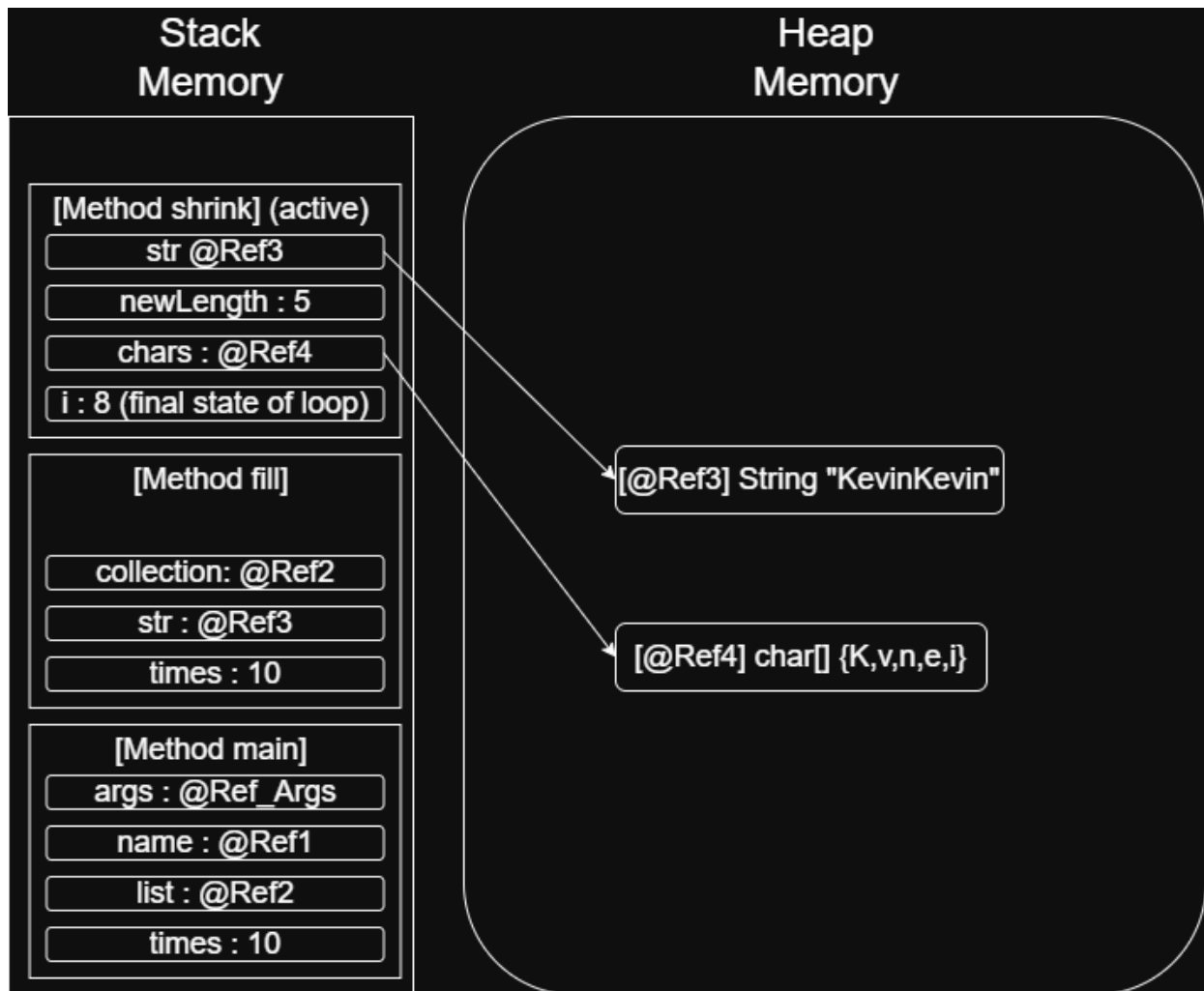


Step 3: Call to shrink Method

Execution:

1. **fill** calls **shrink(str)**.
2. **str** passes reference **@Ref3** ("KevinKevin").
3. Inside **shrink**:
 - **newLength** calculated: $(10 / 2) + 0 = 5$.
 - **char[] chars** array allocated in heap (size 5).
 - **Loop runs:** Extracts characters at indices 0, 2, 4, 6, 8 ('K', 'v', 'n', 'e', 'i').

Diagram:

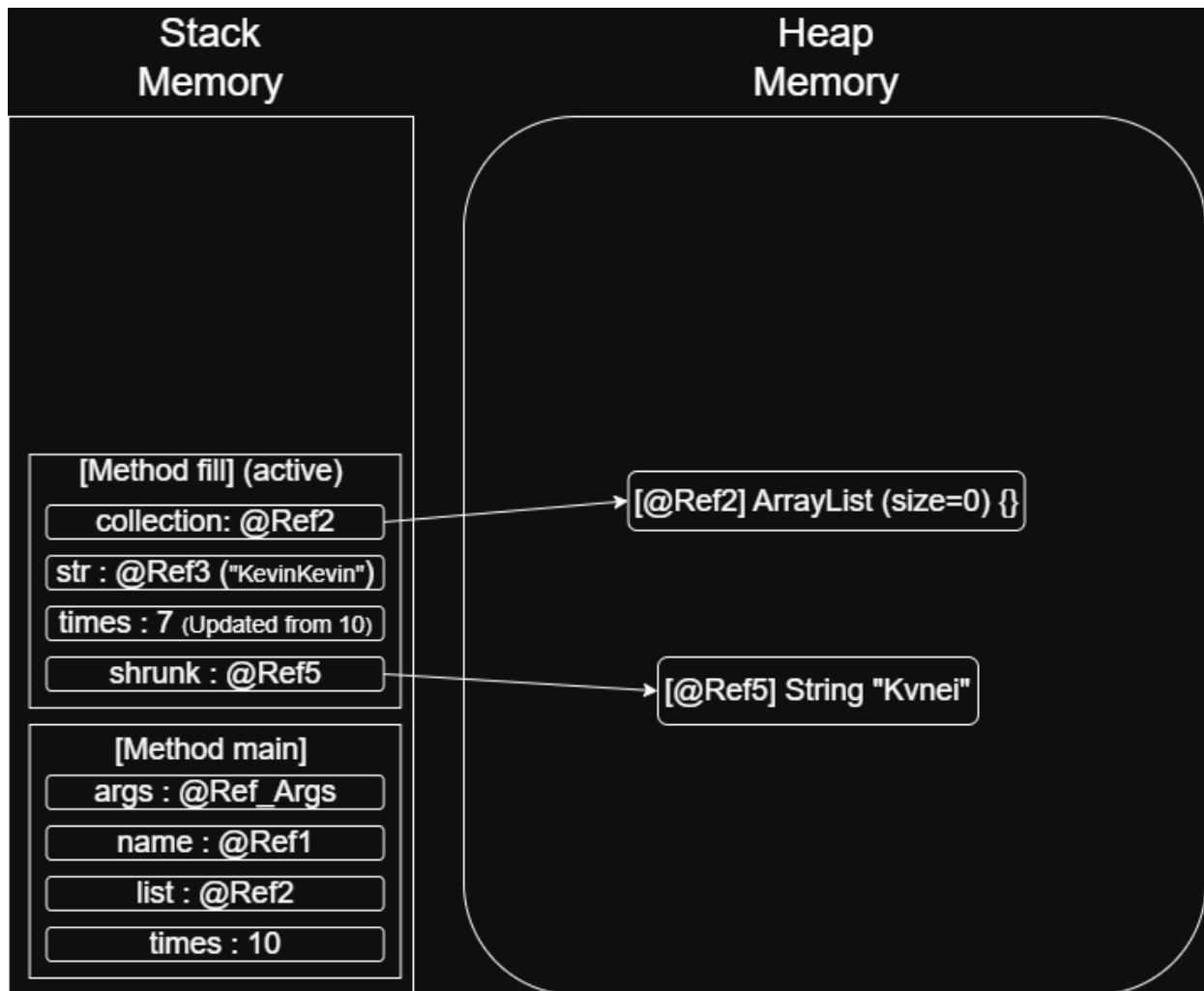


Step 4: shrink Returns to fill

Execution:

- shrink creates a **new String** from the char array: "Kvnei".
- shrink frame is popped from stack.
- In fill:
 - shrunk variable now holds the reference to "Kvnei".
 - times is updated: $(10 + 5) / 2 = 7$. Note: This only changes the local variable times in fill.

Diagram:

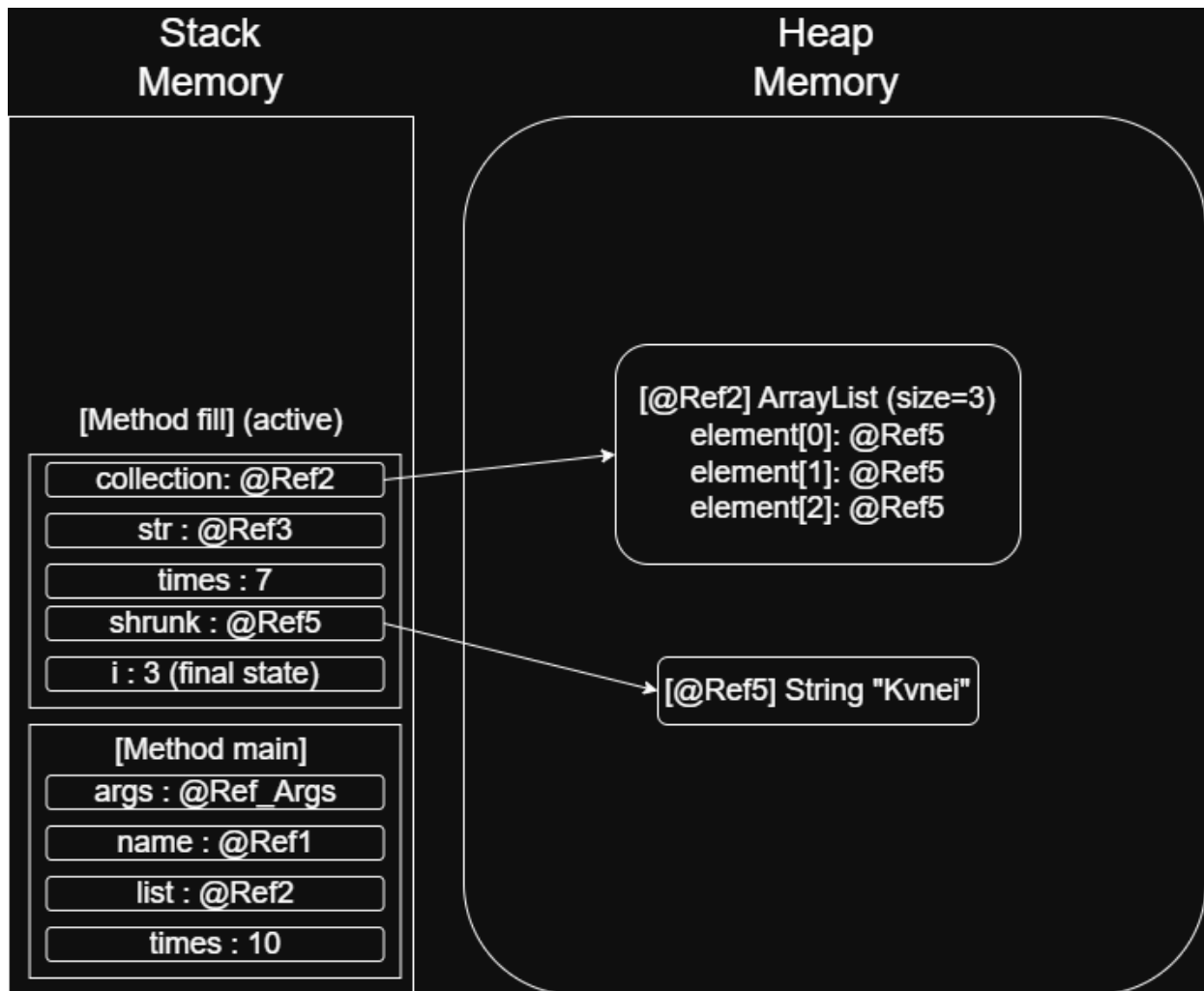


Step 5: fill Loop Execution

Execution:

1. Loop condition: $i < \text{times} / 2 \rightarrow i < 3$ (since $7/2 = 3$).
2. Loop runs for $i = 0, i = 1, i = 2$.
3. `collection.add(shrunk)` adds reference @Ref5 to the ArrayList in the Heap three times.

Diagram:



Step 6: Return to `main` (Final State)

Execution:

- `fill` returns the integer `7`.
- `fill` frame is popped.
- `main` resumes to evaluate `System.out.println`.
- Calculation: `times` (local to `main`, which is `10`) + `7` (returned result) = `17`.
- Note that `list` in `main` still points to `@Ref2`, which now contains the data added by `fill`.

Diagram:

